

Baba in Wonderland: Online Self-Supervised Dynamics Discovery for Executable World Models

SeungWon Seo*

Korea University
Seoul, Republic of Korea
ssw03270@korea.ac.kr

DongHeun Han*

Kyung Hee University
Yongin, Republic of Korea
hand32@khu.ac.kr

SeongRae Noh

Korea University
Seoul, Republic of Korea
rhosunr99@korea.ac.kr

HyeongYeop Kang†

Korea University
Seoul, Republic of Korea
siamiz_hkang@korea.ac.kr

Abstract

Executable world models can be read, edited, executed, and reused for planning, but only if the program captures the environment’s transition law rather than semantic shortcuts in its surface vocabulary. We study online executable world-model learning under prior misalignment, where an agent must induce state-dependent dynamics from interaction evidence alone, without rule descriptions, reward signals, or trustworthy lexical priors. We introduce **Alice**, a closed-loop system that treats failed candidate updates as structural signal: when a candidate explains a new transition but loses previously explained ones, the preservation conflict reveals dynamics that the current program had conflated. Alice refines these conflicts into hypothesis classes that both provide compact, class-stratified preservation counterexamples for update and guide frontier exploration toward transitions that are novel and underrepresented with respect to the current program. We evaluate Alice on *Baba in Wonderland*, a prior-misaligned variant of *Baba Is You* that preserves simulator dynamics while replacing semantically meaningful rule-property labels with unrelated words. Experiments show that Alice substantially improves executable world-model learning under prior misalignment, and ablations show that both class refinement and class-aware exploration contribute.

1 Introduction

Executable world models promise dynamics models that can be read, edited, executed, and reused for planning without post-hoc interpretation [3, 17, 24, 28]. This promise, however, depends on whether the program captures the environment’s transition law. When a language model synthesizes code in an environment whose surface vocabulary aligns with pretrained semantic priors, much of this inductive burden is supplied for free. When it does not, the same vocabulary becomes an active source of error [29, 35, 36], and the editable program, however clean its source, encodes the wrong dynamics.

Recent progress on executable and programmatic world models has largely operated on the favorable side of this divide, relying on curated transition traces, natural-language rule descriptions, or vocabularies whose surface meanings match the dynamics they label [3, 17, 24, 28]. A separate line

*These authors contributed equally to this work.

†Corresponding author.

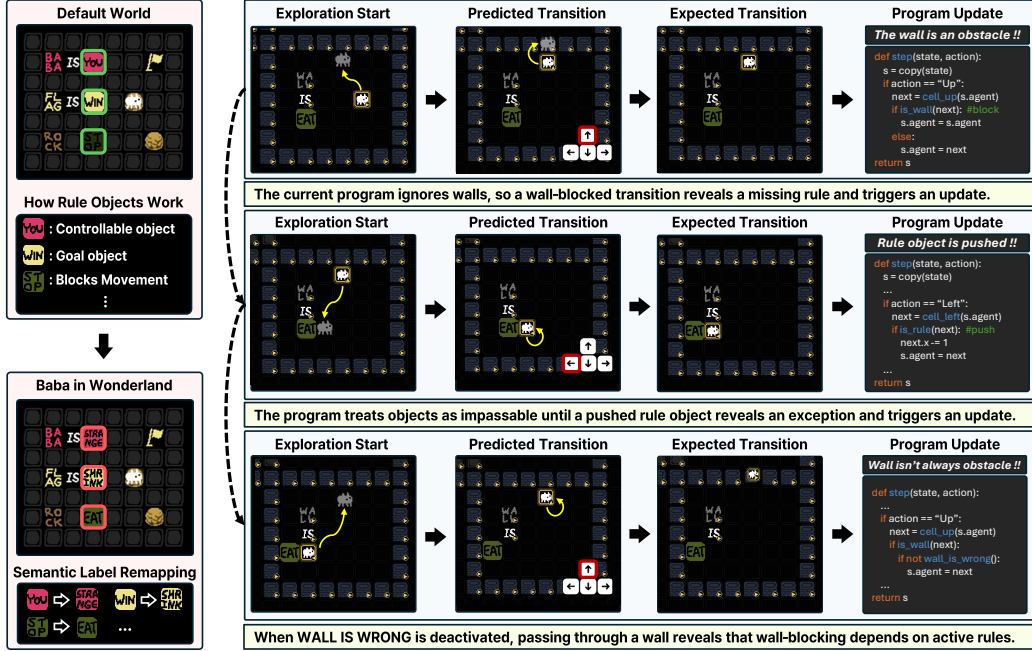


Figure 1: **Alice in Baba in Wonderland.** Left: simulator dynamics are preserved while rule-property labels are semantically remapped, removing lexical shortcuts. Right: exploration exposes prediction errors that drive successive executable world-model revisions.

confronts regimes without such scaffolds, but typically reasons inside the Large Language Model (LLM) rather than synthesizing a persistent program [29, 36]. Their intersection – online executable world-model construction without trustworthy surface vocabulary – has received little direct study, yet this is where executable models are most useful: novel domains, opaque simulators, and unfamiliar terminology. The agent cannot be handed the rules; it must induce them from interaction, and an opaque neural model gives up much of the reason to seek executable form.

We instantiate this regime with **Baba in Wonderland**, a controlled prior-misaligned variant of *Baba Is You*³. The original game already exhibits state-dependent dynamics: rule sentences live inside the state, so the same action can produce different outcomes as rule blocks are rearranged. Yet labels such as STOP, MOVE, and WIN still provide semantic shortcuts. *Baba in Wonderland* preserves the simulator dynamics, action space, and rule-parsing mechanics, but replaces these labels with semantically unrelated words and provides no dictionary. A remapped token remains a stable identifier for the same role, but its surface form no longer hints at how that role behaves; successful modeling must be grounded in transition evidence rather than lexical priors.

Building an executable world model online in this regime exposes two coupled bottlenecks. The first is an *update bottleneck*: each unexplained transition demands a code revision that explains the new transition without losing previously explained ones. Verifying preservation against the full history is conceptually clean but practically infeasible because the dataset grows while the LLM context window does not [20, 21, 33]. Updating from a small subset is context-efficient [34] but risks brittle revisions that fit the target while silently regressing on rare or structurally distinct cases [37]. The question is therefore which subset can expose distinctions that the program itself has not yet made.

The second is an *exploration bottleneck*. Improving the model requires transitions that expose gaps in its current hypotheses, but no external reward, rule description, or pretrained prior identifies which transitions those are [13, 29, 36]. Without such a signal, exploration either spends budget on transitions the current program already explains or traverses the state graph without knowing which transitions matter. What is needed is a notion of “novelty with respect to the program being built”, derived from interaction rather than rewards or priors.

³Steam page for *Baba Is You*.

Our central observation is that both bottlenecks share a hidden source of signal: *the program’s own update failures*. When a candidate revision explains a new transition but loses previously explained ones, the lost transitions are evidence that the earlier program had conflated distinct dynamics under a single branch of code. The preservation conflict reveals a latent partition of accumulated evidence into hypothesis classes that respond differently to the proposed change. Once explicit, this partition also defines what is underrepresented relative to the current program, turning a rejection event into the substrate for both compact update evidence and program-relative exploration.

We realize this idea in **Alice**, a closed-loop system that maintains a single persistent executable program while alternating LLM-based updates with update-guided exploration. Rejected candidate updates refine accumulated transitions into hypothesis classes, which serve two roles: they supply compact, class-stratified preservation counterexamples for update, and they shape an embedding space in which a frontier explorer scores candidates by particle-entropy novelty and class-rarity coverage. Across *Baba in Wonderland* and the original Default World, Alice learns substantially more accurate executable world models than the closest prior interaction-driven baseline, with the gap widening sharply under prior misalignment; ablations confirm that class refinement and class-aware exploration each contribute. Figure 1 illustrates the setting and update loop.

We summarize our contributions as follows:

- **A regime and a benchmark.** We introduce *Baba in Wonderland*, a controlled variant of *Baba Is You* in which simulator dynamics are preserved but surface labels are remapped to semantically unrelated words. The benchmark isolates online executable world-model learning in a regime where lexical priors offer no shortcut.
- **Failed updates as structural signal.** We show that rejected candidate program updates expose a latent partition of accumulated transitions into hypothesis classes, refined monotonically as more updates are attempted. This turns an event normally discarded as a rejection into a reusable artifact for organizing experience.
- **A closed loop that uses this signal twice.** The same hypothesis classes supply class-stratified preservation evidence for the next program update and shape an embedding in which a frontier explorer scores candidates by novelty and class rarity. One signal therefore drives both program updates and data collection.
- **Empirical evidence under prior misalignment.** On *Baba in Wonderland*, Alice substantially outperforms the closest interaction-driven executable world-model baseline; ablations isolate the contribution of class refinement and of each exploration component.

2 Related Work

Learning Executable World Models World-model learning is often studied through neural latent dynamics models [6–9], and recent work also uses LLMs as implicit simulators or reasoning backends for planning [10, 12, 26, 27, 32]. These models can be powerful, but their dynamics are not preserved as a persistent executable artifact that can be inspected, locally updated, and reused as source code. This distinction matters in environments where pretrained semantic priors conflict with the transition law [29, 35, 36]: language-space reasoning can follow surface meanings even when those meanings are misleading or when dynamics vary with state.

The closest line of work asks LLMs to externalize environment knowledge as executable code or structured programmatic world models, including reusable procedures for embodied control [18, 31], interaction-driven code world models [28], search-guided synthesis [3], natural-language-rule-to-code game models [17], and compositional code-based modeling [24]. Our work shares the goal of learning an executable world model, but studies a stricter regime: the agent must maintain a persistent program online from interaction evidence alone, while surface vocabulary is prior-misaligned and the effective transition law changes with the state.

Self-Supervised Exploration for Dynamics Discovery Online world-model learning also requires exploration: the agent must collect transitions that expose gaps in the current hypothesis rather than merely expand a dataset. Task reward can direct data collection in model-based control [8], and solution-finding objectives can guide executable world-model agents [28], but these objectives may be unavailable in reward-free settings [13] or misaligned with dynamics discovery. Intrinsic-motivation

and reward-free methods instead use information gain or uncertainty [11, 15], novelty [2, 22], disagreement [23], memory or directed exploration [1, 4], and entropy or mutual information in behavior and transition spaces [5, 16, 19, 30].

For executable world-model learning, however, generic diversity is not enough: exploration should be relative to the program being updated. Some prior work makes exploration model-relative through learning progress [15] or anticipated model novelty [25], but these signals are designed for neural predictors or downstream control adaptation. Alice instead derives the exploration space from the executable update process itself. Rejected candidate updates induce hypothesis classes over transition evidence, and the Explorer uses these classes to measure novelty and underrepresentation with respect to the current program’s explanatory frontier.

3 Problem Setting

3.1 Formal Setup

We formulate online executable world-model learning in a deterministic Markov decision process (MDP). Let $\mathcal{S}$ and $\mathcal{A}$ denote discrete state and action spaces, and let $T : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ denote the environment transition function. At each global interaction step t , the agent observes a transition $\tau_t = (s_t, a_t, s_{t+1})$ with $s_{t+1} = T(s_t, a_t)$, and accumulates the dataset $D_t = (\tau_1, \tau_2, \dots, \tau_t)$.

Rather than learning a neural predictor, we represent the world model as a source-code program P . We interpret P as an executable hypothesis about T , whose validity is tested only through observed transition evidence: given (s, a) , the program either returns a predicted next state $\hat{s}'$ or fails to execute. We say that P explains a transition $\tau = (s, a, s')$ if it executes successfully on (s, a) and outputs $\hat{s}' = s'$, and we write $E(P; D) = \{\tau \in D \mid P \text{ explains } \tau\}$ for the explained subset of D .

The agent operates under a finite global interaction budget H , and the executable hypothesis is continually revised from the accumulated evidence in D_t . Let P_i denote the current accepted program, with i indexing accepted updates. When a newly observed transition τ_{t+1} is not explained by P_i , the agent proposes a candidate update Q , which is accepted as P_{i+1} only if it explains τ_{t+1} while preserving every previously explained transition: $E(P_i; D_t) \cup \{\tau_{t+1}\} \subseteq E(P_{i+1}; D_{t+1})$. The system-level objective is to use the interaction budget H to expand $E(P_i; D_t)$, and exploration contributes by selecting interactions likely to expose explanatory failures of P_i .

This is not an offline modeling problem over a fixed transition dataset. In our setting, pre-collected traces and rule descriptions are unavailable, and task rewards do not specify which transitions improve the hypothesis. Exploration therefore means collecting online evidence for refining the current executable hypothesis about T , not maximizing external reward.

3.2 Baba in Wonderland

We instantiate this setting in *Baba Is You*, a puzzle game in which rule sentences live inside the grid as movable text blocks. The environment is challenging on two fronts. Its state space is large and combinatorial, so unguided exploration cannot enumerate it within any realistic budget. More fundamentally, because rules are part of the state, the same action can produce different outcomes depending on which sentences are currently active. A correct hypothesis about T must therefore capture how the currently active rule configuration determines transition outcomes — not a fixed background dynamics, but a transition law that the agent’s own actions can rewrite.

The original game, however, has one feature that obscures the difficulty we want to study. Its rule-property labels — STOP, MOVE, WIN — still carry their everyday meanings, so an LLM can often guess correct dynamics from surface vocabulary alone. This residual semantic shortcut conflates two distinct sources of success: induction from transition evidence, and lookup from pretrained priors. To separate them, we introduce *Baba in Wonderland*, a minimal intervention that preserves the simulator dynamics, action space, and rule-parsing mechanics, and changes only the surface labels of `rule_property` tokens, remapping them to semantically unrelated words; see Section A.1 for environment details and the full remapping.

The agent receives no dictionary and no natural-language explanation of the remapping. The role of each remapped token must therefore be inferred from transition evidence and from how the token participates in active rule sentences, rather than from pretrained lexical priors alone.

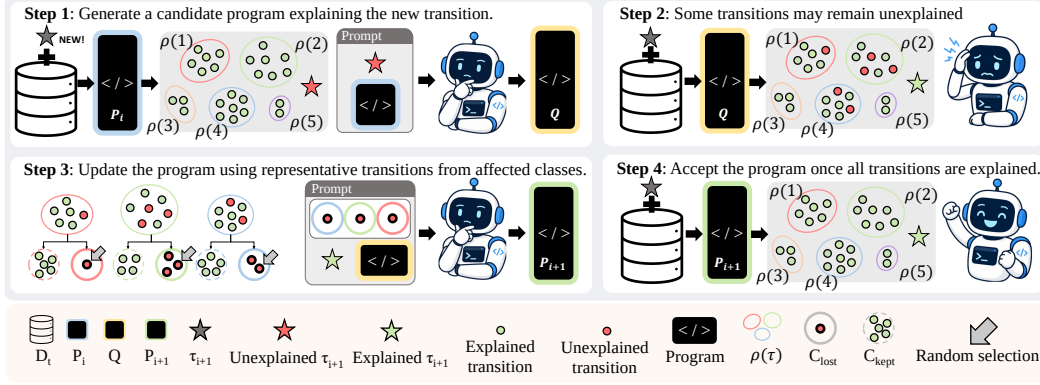


Figure 2: Hypothesis-class refinement. Rejected target-explaining updates reveal lost transitions, split affected classes, and provide preservation counterexamples for later retries.

4 Method

Section 3 identified two coupled bottlenecks for online executable world-model learning: an *update bottleneck*, in which each candidate revision must preserve every previously explained transition under a context budget that cannot hold all of D_t , and an *exploration bottleneck*, in which no external signal indicates which transitions would expose gaps in the current hypothesis. We propose **Alice**, a closed-loop system that addresses both by treating the same artifact — failed candidate updates — as a structural signal: the conflicts that arise when a candidate gains τ_{t+1} but loses some previously explained transitions reveal which transitions belonged to distinct hypotheses all along.

The system has two components that share this signal. The **Programmer** maintains the persistent program P_i and uses preservation conflicts to organize D_t into hypothesis classes that supply compact counterexamples for the next update. The **Explorer** uses the same classes to score frontier candidates by both novelty and class underrepresentation, steering data collection toward the program’s explanatory frontier. The two components operate in a closed loop, alternating program updates with update-guided exploration over the agent’s interaction budget H .

4.1 Refining Hypothesis Classes from Failed Updates

When τ_{t+1} is unexplained by P_i , the Programmer asks an LLM to propose a candidate update Q that explains τ_{t+1} while preserving every transition in $E(P_i; D_t)$. Verifying preservation is straightforward: re-run Q on every transition in D_t and check that the explained set still contains $E(P_i; D_t)$. *Generating* a candidate that will pass this check is harder, because the prompt must convey what those transitions actually look like, and neither P_i nor τ_{t+1} alone is enough. P_i is an executable hypothesis induced from D_t , not a lossless record of it: its code specifies how to compute next states without recording which transitions support each branch. The evidence must therefore come from D_t itself — but D_t grows online while the prompt budget is fixed, so the Programmer cannot include it in full. The Programmer therefore selects a compact reference set $R_t \subseteq D_t$ to include in the prompt. The full inductive program-update prompt template is provided in Section A.2. The question is which transitions should be in R_t .

The answer is that the LLM itself can produce this evidence whenever a candidate is rejected. Suppose Q explains τ_{t+1} but fails to preserve some previously explained transitions. We call the set of lost transitions the *preservation counterexamples* of Q ,

$$L_{pc}(Q; P_i, D_t) = E(P_i; D_t) \setminus E(Q; D_t).$$

These are transitions that P_i explained, grouping them with whatever rule its code applies in the relevant branch, yet Q now explains them differently from τ_{t+1} . The merger inside P_i was therefore hiding a distinction, and L_{pc} surfaces exactly the transitions where that distinction matters. Each rejected candidate that explains its target thus contributes one cut through $E(P_i; D_t)$, separating transitions that respond differently to the proposed update. The Programmer accumulates these cuts as a mutable collection of *hypothesis classes* $\mathcal{C}$ over $E(P_i; D_t)$. Transitions remain in the same class until separated by a rejected candidate (Figure 2).

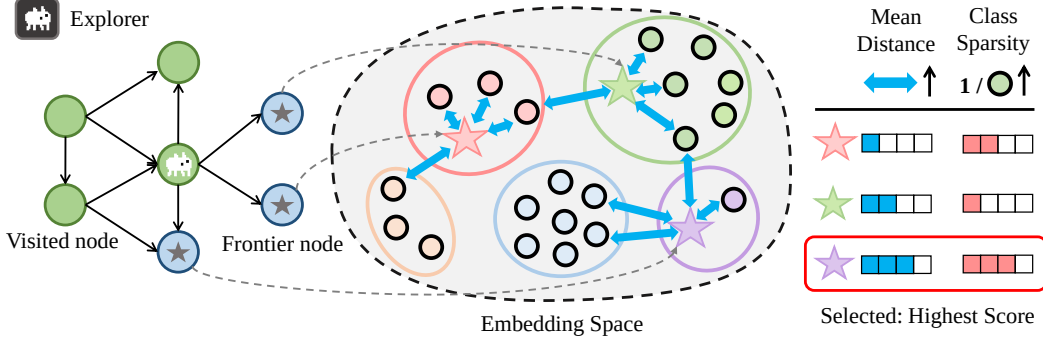


Figure 3: Update-guided frontier scoring. The Explorer selects frontier candidates by combining embedding novelty with expected coverage of underrepresented hypothesis classes.

We initialize $\mathcal{C}$ from the accepted update sequence itself. Transitions that entered the explained set under the same bundle of hypotheses tend to provide redundant preservation evidence, so we group them by the earliest accepted version from which they have remained explained:

$$\rho(\tau) = \min\{j \leq i \mid \tau \in E(P_k; D_t) \text{ for all } k = j, \dots, i\}.$$

For example, if τ is explained by P_1 , lost by P_2 , and explained by every version from P_3 onward, then $\rho(\tau) = 3$. Transitions sharing $\rho(\tau)$ form the *root classes* of $\mathcal{C}$.

Root classes alone are coarse, however, because an early update can explain its target with a rule broader than the truth, sweeping in transitions that should later prove to obey something different. Rejected candidates expose such overgeneralizations at the class level. When $L_{\text{pc}}(Q; P_i, D_t)$ covers only part of a class C , the candidate Q separates C into a preserved subset and a lost subset,

$$C_{\text{lost}} = C \cap L_{\text{pc}}(Q; P_i, D_t), \quad C_{\text{kept}} = C \setminus C_{\text{lost}},$$

giving evidence that C had merged transitions responding differently to the new hypothesis. Whenever both subsets are nonempty, the Programmer replaces C with the two refined classes and stores Q as the split-inducing test that will route future transitions to the appropriate side. A refined class can be split again by later rejected candidates, making $\mathcal{C}$ monotonically finer over retries.

The compact reference set R_t is then drawn from L_{pc} by stratification across $\mathcal{C}$: the Programmer groups the preservation counterexamples by their refined classes, picks up to $n = 3$ classes, and samples $m = 1$ transition from each. These few transitions, each witnessing a distinct preservation constraint, replace what would otherwise be random samples from D_t . The acceptance check itself remains over the full $E(P_i; D_t)$, so R_t shapes only the update context, not the correctness criterion. After Q is accepted, τ_{t+1} and later explained transitions inherit the existing refinements: they are routed first by ρ into their root class, then by the stored split candidates into the appropriate refined class, so newly explained transitions immediately contribute evidence to the same structure that produced them. The same $\mathcal{C}$ also guides how the Explorer collects future transitions in Section 4.2.

4.2 Update-Guided Exploration

The Programmer turns transitions into program updates; the Explorer must turn the interaction budget H into the informative transitions. A transition is informative for the next program update if two conditions hold: it should expose a failure of the current program P_i so that an update is triggered, and ideally it should also induce a new hypothesis class so that $\mathcal{C}$ refines further. Neither property is observable before the candidate is executed. The Explorer therefore maintains a transition graph over reached states and scores each frontier candidate $x = (s, a)$, where s is reached but unexpanded, using two pre-execution proxies: *embedding novelty*, which favors candidates lying far from D_t in a learned state-action space, and *expected class coverage*, which favors candidates likely to fall into hypothesis classes with little existing evidence. Together, these proxies bias graph-frontier exploration toward the program’s explanatory frontier. Figure 3 illustrates the procedure.

For embedding novelty to track the program’s explanatory frontier rather than mere surface variation, candidates that fall in the same hypothesis class should map to nearby embeddings, and candidates

from different classes should map to distant ones. The hypothesis classes $\mathcal{C}$ provide exactly the supervision needed for this geometry: the Explorer trains $h_\theta(x)$ with the supervised contrastive loss of Khosla et al. [14], pulling together embeddings within the same class and pushing apart embeddings across classes. Supervision is limited to active classes $\mathcal{C}_{\text{act}} \subseteq \mathcal{C}$ with enough transitions for stable positives; low-count classes remain in $\mathcal{C}$ but are excluded.

Given this embedding, novelty is measured by a k -nearest-neighbor particle-entropy score [19],

$$r_h(x) = \log \left(1 + \frac{1}{k} \sum_{\tilde{h} \in \mathcal{N}_k(h_\theta(x); \mathcal{B}_h)} \|h_\theta(x) - \tilde{h}\|_2 \right),$$

where $\mathcal{B}_h$ is the embedding set induced by D_t and $\mathcal{N}_k$ returns the $k = 32$ nearest neighbors. Candidates far from previously observed embeddings score high.

The second proxy, *expected class coverage*, biases exploration toward classes with little existing evidence. Using the same active class set, each $C \in \mathcal{C}_{\text{act}}$ contributes a class-count self-information $u(C) = -\log(|C|/N)$, where $N = \sum_{C \in \mathcal{C}_{\text{act}}} |C|$ is the total count across active classes; rare classes have larger $u(C)$. Since the class label of an unexecuted candidate is itself unobservable, the Explorer estimates it from class prototypes — the mean embedding of explained transitions assigned to each class. A softmax over cosine similarities to prototype embeddings yields a pre-execution class distribution $q_\theta(C | h_\theta(x))$, and the expected class-coverage score is

$$r_C(x) = \sum_{C \in \mathcal{C}_{\text{act}}} q_\theta(C | h_\theta(x)) u(C).$$

The two proxies combine into the frontier score

$$S_{\text{frontier}}(x) = \lambda_h r_h(x) + \lambda_C r_C(x).$$

with $\lambda_h = 1$ and $\lambda_C = 0.05$ throughout our experiments. Each collection batch greedily selects the highest-scoring candidates from the current frontier under the interaction budget.

Executed transitions feed back into the loop. Explained transitions sharpen the class-structured evidence used to train h_θ , while unexplained transitions become the Programmer’s next explanatory failures. The single signal of failed updates therefore drives both halves of Alice: it shapes which counterexamples the LLM sees in R_t , and it shapes which transitions the agent seeks through S_{frontier} . Section A.3 relates this score to an entropy–mutual-information objective.

5 Experiments

We evaluate Alice along two axes that jointly support the claims in Section 1: (i) end-to-end accuracy of the learned executable world model under prior misalignment, both for online interaction on regular levels and for held-out generalization to extra levels; and (ii) ablations isolating the contribution of each component and the synergy between them. Robustness to LLM backbones, additional diagnostic experiments, and qualitative examples are reported in Section A.4 and Section A.5.

5.1 Experimental Setup

Settings. We evaluate executable world-model learning on levels drawn from *Baba Is You*, comprising 32 regular levels and 8 extra levels. Each level is instantiated under two label settings: *Default World*, which uses the original `rule_property` labels, and *Baba in Wonderland*, which preserves the simulator dynamics but replaces the surface labels of `rule_property` tokens with semantically unrelated labels (Table 5). The agent is given no dictionary and no description of the remapping. Online experiments collect interaction data on regular levels and evaluate on human-play solution transitions from the same levels; offline experiments train on regular-level transitions and evaluate on held-out extra-level transitions. Additional protocol details appear in Section A.6.

Metrics. *All Acc.* is the fraction of evaluated transitions for which the predicted next state exactly matches the environment next state. *Balanced Acc.* applies the same exact-match criterion on a class-reduced evaluation set in which transitions are grouped by action and canonical state-difference signature, with one representative sampled per group; details appear in Section A.7. Compile failures

Table 1: Online interaction results on regular levels. All Acc. and Balanced Acc. are exact one-step transition accuracies; Calls reports learning-time LLM calls.

Method	Default World			Baba in Wonderland		
	All Acc. $\uparrow$	Balanced Acc. $\uparrow$	Calls	All Acc. $\uparrow$	Balanced Acc. $\uparrow$	Calls
WorldCoder	0.920	0.862	97	0.568	0.198	57
Alice	0.993	0.992	26	0.982	0.973	100

Table 2: Offline fixed-data results. Methods train on regular-level solution transitions and evaluate on held-out extra-level transitions.

Method	Default World			Baba in Wonderland		
	All Acc. $\uparrow$	Balanced Acc. $\uparrow$	Calls $\downarrow$	All Acc. $\uparrow$	Balanced Acc. $\uparrow$	Calls $\downarrow$
GIF-MCTS	0.866	0.799	100	0.704	0.628	100
CWM	0.962	0.928	100	0.826	0.734	100
Alice	1.000	1.000	22	0.956	0.932	100

and runtime errors are counted as incorrect predictions. We additionally report *Calls*, the cumulative number of LLM calls used during learning. For online runs each method is capped at 100 total LLM calls and 15 calls per single update iteration; offline runs are similarly capped at 100 total LLM calls.

Baselines. For online experiments, we compare against **WorldCoder** [28], the closest prior interaction-driven executable world-model learner. Like Alice, WorldCoder maintains executable code as its world model, but it does not organize failed program updates into hypothesis classes for later preservation or exploration. For offline fixed-data experiments, we additionally compare against **GIF-MCTS** [3] and **CWM** [17], which together cover search-based and population-based code world-model synthesis. All baselines use the same LLM backbone (GPT-5.4) and the same LLM-call caps as Alice; in the offline comparison Alice trains only on the fixed dataset and does not use its Explorer. Per-run costs are substantial in our regime, so unless otherwise noted each reported number reflects one run with a fixed configuration.

5.2 Executable World-Model Learning

Online Interaction. Table 1 reports online performance on regular levels in both label settings. In *Default World*, WorldCoder remains relatively strong but trails Alice in both All Acc. and Balanced Acc., reflecting that semantic priors help with many common transitions but can still miss rare or compositional dynamics. The gap widens sharply in *Baba in Wonderland*, where lexical priors are unreliable: WorldCoder’s All Acc. drops by more than 0.35 and its Balanced Acc. collapses below 0.20, while Alice retains 0.98 All Acc. and 0.97 Balanced Acc.

The WorldCoder call counts deserve attention. In *Baba in Wonderland*, WorldCoder consumes only 57 calls not because it is more efficient but because its update loop terminates: a candidate update that fits the most recent failure regresses on previously explained transitions, the per-iteration retry budget is exhausted, and no further update is accepted. Alice avoids this failure mode because its hypothesis classes supply the LLM with class-stratified preservation counterexamples that target the constraints a naive update tends to violate.

Fixed-Data Comparison. Table 2 compares Alice with GIF-MCTS, and CWM when all methods learn from the same fixed regular-level transitions and are evaluated on held-out extra-level transitions. In *Default World*, semantically aligned surface labels make executable-rule synthesis easier for all methods, and Alice reaches perfect held-out accuracy using fewer than a quarter of the LLM calls. CWM is the strongest baseline in this setting, yet its remaining Balanced Acc. gap indicates that high overall accuracy can still miss less frequent transition types. The comparison becomes more diagnostic in *Baba in Wonderland*: because no method can collect additional transitions from the unseen extra levels, held-out accuracy depends on how effectively each method converts the fixed train evidence into executable dynamics without relying on lexical shortcuts. Here Alice improves over

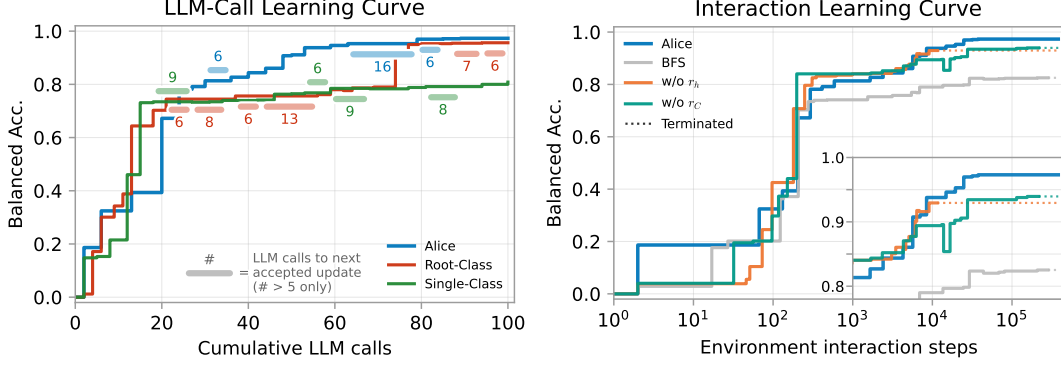


Figure 4: Left: hypothesis-class ablations over cumulative LLM calls; bars show calls before the next accepted update (> 5 only). Right: exploration ablations over environment interaction steps.

both GIF-MCTS and CWM, with the largest margin in Balanced Acc., showing that the persistent update loop better preserves rare or easily conflated dynamics under prior misalignment.

5.3 Ablation Study Results

Hypothesis classes. We ask how much structure is needed for the Programmer to turn preservation counterexamples into accepted program updates. **Single-Class** collapses all explained transitions into one class, while **Root-Class** keeps only the root classes induced by $\rho(\tau)$, before any refinement from rejected candidates. The left panel of Figure 4 makes the update bottleneck visible: after easy early updates, Single-Class repeatedly spends LLM calls before acceptance and saturates, since R_t is drawn from an unstructured pool of redundant preservation evidence. Root-Class partly relieves this bottleneck through the accepted-update partition, explaining its late improvement, but its mid-run retry bars show that coarse root classes can still conflate distinct dynamics after an overgeneralized update. Alice closes this gap by turning rejected updates into preservation conflicts that refine hypothesis classes, yielding a compact, class-stratified R_t . This makes Alice’s middle segment more productive, with longer retry bars concentrated near high accuracy on rare, structurally specific dynamics.

Exploration objective. We ask whether the Explorer must actively shape which transition evidence is collected before each program update. **BFS** selects frontier candidates in breadth-first order, removing program-relative frontier scoring; **w/o r_h** and **w/o r_C** remove the embedding-novelty and expected-class-coverage terms from S_{frontier} , respectively. In Figure 4, BFS is the least efficient because it expands the reachable graph without prioritizing transitions near the current explanatory frontier. The score ablations expose complementary failures: without r_h , expected class coverage remains available but does not expand the learned state-action space sufficiently; without r_C , embedding novelty expands outward but can over-collect variants within already well-represented classes, leaving rarer hypothesis classes under-covered. The early- 10^4 -step drop in **w/o r_C** illustrates the consequence: an accepted update can fit a narrow new case when D_t lacks enough preservation evidence to expose the resulting overgeneralization as a preservation conflict. The embedding space t-SNE visualization in Figure 6 gives the corresponding qualitative view. Alice avoids both failures by using r_h for embedding space coverage and r_C to distribute that coverage across hypothesis classes, yielding more reliable program-relative exploration.

6 Conclusion

We studied online executable world-model learning under prior misalignment, where state-dependent dynamics must be induced from interaction evidence rather than rule descriptions, rewards, or trustworthy lexical priors. Alice maintains a persistent executable program and turns failed candidate updates into hypothesis classes that support both class-stratified program update and program-relative frontier exploration. Through *Baba in Wonderland*, we show that this closed loop can ground executable world-model construction in transition evidence even when surface labels are misleading.

However, our experiments are limited to deterministic, discrete, symbolic environments where transition explainability can be checked exactly by executing a program. Extending the idea to stochastic, continuous, or partially observable domains will require broader notions of explanation, preservation, and update-induced evidence.

References

- [1] Adrià Puigdomènech Badia, Pablo Sprechmann, Alex Vitvitskyi, Daniel Guo, Bilal Piot, Steven Kapturowski, Olivier Tieleman, Martín Arjovsky, Alexander Pritzel, Andrew Bolt, et al. Never give up: Learning directed exploration strategies. *arXiv preprint arXiv:2002.06038*, 2020.
- [2] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. *arXiv preprint arXiv:1810.12894*, 2018.
- [3] Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by monte carlo tree search. *Advances in Neural Information Processing Systems*, 37:60429–60474, 2024.
- [4] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O Stanley, and Jeff Clune. Go-explore: a new approach for hard-exploration problems. *arXiv preprint arXiv:1901.10995*, 2019.
- [5] Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function. *arXiv preprint arXiv:1802.06070*, 2018.
- [6] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3):440, 2018.
- [7] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pages 2555–2565. PMLR, 2019.
- [8] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [9] Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*, 2023.
- [10] Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 8154–8173, 2023.
- [11] Rein Houthooft, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. Vime: Variational information maximizing exploration. *Advances in neural information processing systems*, 29, 2016.
- [12] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR, 2022.
- [13] Chi Jin, Akshay Krishnamurthy, Max Simchowitz, and Tiancheng Yu. Reward-free exploration for reinforcement learning. In *International Conference on Machine Learning*, pages 4870–4879. PMLR, 2020.
- [14] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschiot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. In *Advances in Neural Information Processing Systems 33*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/d89a66c7c80a29b1bdbab0f2a1a94af8-Abstract.html>.
- [15] Kuno Kim, Megumi Sano, Julian De Freitas, Nick Haber, and Daniel Yamins. Active world model learning with progress curiosity. In *International conference on machine learning*, pages 5306–5315. PMLR, 2020.

- [16] Michael Laskin, Hao Liu, Xue Bin Peng, Denis Yarats, Aravind Rajeswaran, and Pieter Abbeel. Unsupervised reinforcement learning with contrastive intrinsic control. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 34478–34491. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/debf482a7dbdc401f9052dbe15702837-Paper-Conference.pdf.
- [17] Wolfgang Lehrach, Daniel Hennes, Miguel Lazaro-Gredilla, Xinghua Lou, Carter Wendelken, Zun Li, Antoine Dedieu, Jordi Grau-Moya, Marc Lanctot, Atil Iscen, et al. Code world models for general game playing. *arXiv preprint arXiv:2510.04542*, 2025.
- [18] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*, pages 9493–9500. IEEE, 2023.
- [19] Hao Liu and Pieter Abbeel. Behavior from the void: Unsupervised active pre-training. *Advances in Neural Information Processing Systems*, 34:18459–18473, 2021.
- [20] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.
- [21] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. Agentboard: An analytical evaluation board of multi-turn llm agents. *Advances in neural information processing systems*, 37:74325–74362, 2024.
- [22] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pages 2778–2787. PMLR, 2017.
- [23] Deepak Pathak, Dhiraj Gandhi, and Abhinav Gupta. Self-supervised exploration via disagreement. In *International conference on machine learning*, pages 5062–5071. PMLR, 2019.
- [24] Wasu Top Piriyakulkij, Yichao Liang, Hao Tang, Adrian Weller, Marta Kryven, and Kevin Ellis. Poe-world: Compositional world modeling with products of programmatic experts. *arXiv preprint arXiv:2505.10819*, 2025.
- [25] Ramanan Sekar, Oleh Rybkin, Kostas Daniilidis, Pieter Abbeel, Danijar Hafner, and Deepak Pathak. Planning to explore via self-supervised world models. In *International conference on machine learning*, pages 8583–8592. PMLR, 2020.
- [26] SeungWon Seo, SeongRae Noh, Junhyeok Lee, SooBin Lim, Won Hee Lee, and HyeongYeop Kang. Reveca: Adaptive planning and trajectory-based validation in cooperative language agents using information relevance and relative proximity. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 23295–23303, 2025.
- [27] SeungWon Seo, SooBin Lim, SeongRae Noh, Haneul Kim, and HyeongYeop Kang. From assumptions to actions: Turning llm reasoning into uncertainty-aware planning for embodied agents. *arXiv preprint arXiv:2602.04326*, 2026.
- [28] Hao Tang, Darren Key, and Kevin Ellis. Worldcoder, a model-based llm agent: Building world models by writing code and interacting with the environment. *Advances in Neural Information Processing Systems*, 37:70148–70212, 2024.
- [29] Xiaojuan Tang, Jiaqi Li, Yitao Liang, Song-chun Zhu, Muhan Zhang, and Zilong Zheng. Mars: Situated inductive reasoning in an open-world environment. *Advances in Neural Information Processing Systems*, 37:17830–17869, 2024.
- [30] Shanchuan Wan, Yujin Tang, Yingtao Tian, and Tomoyuki Kaneko. Deir: efficient and robust exploration through discriminative-model-based episodic intrinsic rewards. *arXiv preprint arXiv:2304.10770*, 2023.

- [31] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [32] Ruoyao Wang, Graham Todd, Ziang Xiao, Xingdi Yuan, Marc-Alexandre Côté, Peter Clark, and Peter Jansen. Can language models serve as text-based world simulators? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1–17, 2024.
- [33] Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, et al. Resum: Unlocking long-horizon search intelligence via context summarization. *arXiv preprint arXiv:2509.13313*, 2025.
- [34] Chunqiu Steven Xia and Lingming Zhang. Keep the conversation going: Fixing 162 out of 337 bugs for \$0.42 each using chatgpt. *arXiv preprint arXiv:2304.00385*, 2023.
- [35] Manjie Xu, Isabella Yin, Xinyi Tu, Chi Zhang, and Yixin Zhu. Code over words: Overcoming semantic inertia via code-grounded reasoning. *arXiv preprint arXiv:2601.18352*, 2026.
- [36] Siyu Zhou, Tianyi Zhou, Yijun Yang, Guodong Long, Deheng Ye, Jing Jiang, and Chengqi Zhang. Wall-e 2.0: World alignment by neurosymbolic learning improves world model-based llm agents. *arXiv preprint arXiv:2504.15785*, 2025.
- [37] Xin Zhou, Bowen Xu, Kisub Kim, DongGyun Han, Thanh Le-Cong, Junda He, Bach Le, and David Lo. Patchzero: Zero-shot automatic patch correctness assessment. *arXiv preprint arXiv:2303.00202*, 2023.

A Appendix

A.1 Baba Is You

Game Rules and Dynamics. Baba Is You is a rule-manipulation puzzle game in which object behavior is determined by textual rules assembled directly in the grid world. Rules become active when text blocks form valid horizontal or vertical rule sentences, and any change in that arrangement can activate, deactivate, or modify the effective dynamics of the environment. Because these rule sentences can be created, broken, moved, or rearranged by the agent’s actions, the currently active rule set is itself part of the environment state rather than a fixed background specification. As a result, predicting the next state requires more than tracking the visible arrangement of objects: the same action can lead to different outcomes depending on which rules are currently active. This makes the game a particularly challenging environment for world-model learning, because its transition law is state-dependent and symbolic rather than fixed and purely geometric. In *Default World*, for example, changing whether a sentence such as WALL IS STOP is active can immediately alter whether walls behave as obstacles, even when the surrounding object layout remains unchanged. Baba Is You therefore provides a natural testbed for executable world-model learning in environments where accurate prediction depends on maintaining explicit hypotheses about mutable rules.

Table 3: Token vocabulary used in the *Baba Is You* benchmark environment.

Type	Vocabulary
rule_noun	ALGAE, BABA, BOG, BOLT, BRICK, BUBBLE, COG, CRAB, DOOR, FLAG, FLOWER, GRASS, HEDGE, ICE, JELLY, KEKE, KEY, LAVA, LOVE, PILLAR, PIPE, REED, ROCK, ROBOT, SKULL, STAR, TEXT, TILE, WALL, WATER
rule_operator	AND, IS
rule_property	DEFEAT, FLOAT, HOT, MELT, MOVE, OPEN, PUSH, SHUT, SINK, STOP, WIN, YOU
world_object	ALGAE, BABA, BOG, BOLT, BRICK, BUBBLE, COG, CRAB, DOOR, FLAG, FLOWER, GRASS, HEDGE, ICE, JELLY, KEKE, KEY, LAVA, LOVE, PILLAR, PIPE, REED, ROCK, ROBOT, SKULL, STAR, TILE, WALL, WATER

Table 4: Rule properties and their roles in the simulator used by the benchmark.

Property	Role
YOU	Makes an object controllable during the agent-driven movement stage.
WIN	Triggers success when a YOU object overlaps a winning object on the same float layer after interaction effects are resolved.
STOP	Blocks entry and movement through the object.
PUSH	Allows the object to be pushed by movers.
MOVE	Causes the object to move automatically each turn, reversing direction if blocked.
DEFEAT	Destroys overlapping YOU objects on the same float layer.
SINK	Removes both itself and other overlapping same-layer objects.
HOT	Destroys overlapping MELT objects on the same float layer.
MELT	Is removed when overlapping a HOT object on the same float layer.
OPEN	Cancels with SHUT, removing both on overlap.
SHUT	Cancels with OPEN, removing both on overlap.
FLOAT	Restricts many overlap-triggered interactions to objects on the same float layer.

Vocabulary. We use four token types throughout the benchmark: `rule_noun`, `rule_operator`, `rule_property`, and `world_object`. Table 3 summarizes the vocabulary used by each type, and Table 4 summarizes the rule properties and their roles in the simulator. Rule nouns and world objects share most of their lexical stems, but they play different semantic roles: the former appear as text blocks in rule sentences, whereas the latter are instantiated entities in the grid world. Among rule operators, IS links an object noun to a property or another noun, while AND allows conjunctions on either side of a rule, enabling forms such as BABA AND ROCK IS YOU and BABA IS YOU AND PUSH. Crucially, rule properties are not merely labels. They alter movement, collision, overlap, destruction, and goal conditions, and multiple active properties can interact within the same turn. These interactions are also order-sensitive. In the original game, their effects are resolved over an ordered update procedure rather than all at once. Broadly, movement-related properties such as

Table 5: Rule-property remapping used in *Baba in Wonderland*. Simulator semantics remain unchanged.

Original	Wonderland	Original	Wonderland	Original	Wonderland	Original	Wonderland
DEFEAT	WAKE	FLOAT	WRONG	HOT	GRIN	MELT	CURIOUS
MOVE	DRINK	OPEN	MAD	PUSH	GROW	SHUT	LATE
SINK	BEGIN	STOP	EAT	WIN	SHRINK	YOU	STRANGE

YOU, MOVE, and PUSH shape how objects are displaced, whereas overlap-triggered properties such as DEFEAT, SINK, HOT/MELT, OPEN/SHUT, and WIN are checked later in the turn. WIN is evaluated especially late, so earlier destructive effects can prevent a win that would otherwise occur. FLOAT further constrains many of these interactions by requiring the relevant objects to occupy the same float layer. As a result, learning an accurate executable world model requires not only inferring which properties are currently active, but also capturing how their effects compose under the simulator’s execution order.

State and Action Representation. We provide states to the LLM as structured JSON rather than as free-form natural-language descriptions. Each state contains the top-level fields `grid_size`, `step`, and `objects`. `grid_size` gives the width and height of the playable interior grid, after trimming the immutable outer border used by the underlying simulator. The `step` field records episode-level status information and currently includes a Boolean `terminated` flag. Each entry in `objects` is a dictionary with keys such as `type`, `word`, and `position`, and directional objects additionally include a `direction` field. Positions are represented as zero-indexed $[x, y]$ coordinates in the trimmed playable grid. Because multiple objects may occupy the same grid cell, overlap is represented by multiple object entries sharing the same `position`. World objects are directional entities in the underlying simulator, and serialized entries expose this through a `direction` field when that orientation is available. For example, a text block may be represented as `{"type": "rule_noun", "word": "baba", "position": [0,0]}`, while a `world_object` entry may additionally include a field such as `"direction": "facing right"`.

Actions are represented by a small discrete set of named commands, `idle`, `up`, `right`, `down`, and `left`. Transitions are therefore recorded as structured triples consisting of a source-state JSON string, an action name, and a next-state JSON string.

Baba in Wonderland Remapping. The central intervention in *Baba in Wonderland* is a remapping of the surface labels attached to `rule_property` tokens. This remapping changes only the words that appear in serialized states and transition prompts. It does not alter the underlying simulator dynamics, the action space, or the mechanics by which rules are parsed and executed. As a result, two environments that differ only by this remapping induce the same transition law even though they present different textual cues to the LLM. The concrete remapping used in our experiments is listed in Table 5.

The purpose of this intervention is to break the direct alignment between pretrained lexical priors and the true behavioral meaning of rule-property tokens. A model can no longer rely on the everyday meaning of an observed label to guess that it should behave like YOU, WIN, or STOP. Instead, it must infer the role of each remapped token from transition evidence and from how that token participates in active rule sentences over time. The agent is given no natural-language explanation of the remapping and no auxiliary dictionary that translates remapped labels back to their original meanings.

This design preserves the combinatorial and state-dependent structure of *Baba Is You* while introducing prior misalignment directly into the observable rule vocabulary. Success in *Baba in Wonderland* therefore provides stronger evidence that the learned executable world model is grounded in interaction evidence rather than in semantically aligned textual shortcuts.

A.2 Prompt Templates

Figure 5 shows the inductive program-update prompt template used by the Programmer. The template provides the current program, the target explanatory failure, and compact preservation evidence selected from hypothesis classes.

Inductive Program Update Prompts

You are preparing a new JSON patch for a Python world model program.

Your task:

- Base your reasoning on the structured state representation and the observed transition differences.
- Use the target failures as the primary objective.
- Edit only the accepted baseline program shown below.
- Prefer repairs that improve structural simplicity by reusing existing logic and minimizing special-case conditions.
- Keep `'predict_next_state'` as the top-level orchestrator of the transition pipeline.
- Revise, replace, consolidate, split, or delete functions according to the general rule they should represent.

`$additional_instructions_block`
`$sandbox_runtime_block`
Accepted baseline program to patch now:

- This is the only program you may edit.

```
'''python
$current_source
'''
```

Accepted baseline target failures to solve with the next patch:

- These are the direct failures the next patch must fix.
- Any field named `accepted_baseline_predicted_next_state` is the accepted baseline output.

`$failed_text`

`$previous_patch_context_block``$regression_witness_block``$previous_smoke_errors_block``$current_errors_block`
Generate exactly one patch that edits the accepted baseline program.

Patch schema (STRICT):

- Return one JSON object in this exact shape:

```
{
  "edits": [
    {"op": "replace"|"add", "name": "function_name", "code": "def ..."}
  ],
  "delete_functions": ["optional_name"]
}
```

Rules:

- Edit only the accepted baseline program shown above.
- Output exactly one JSON object matching the patch schema.
- Do not use import statements, comments, or docstrings.
- Keep function names stable, and ensure `'predict_next_state(state, action)'` exists and returns a dict.
- Avoid runtime errors using explicit type and bounds checks.
- Do not use `try/except`, `with`, `class`, `lambda`, `global`, `nonlocal`, `async/await`, or `yield`.

Patch semantics:

- Each edit must provide one complete Python function definition in `'code'`, not a body fragment.
- Multiple edits apply in order.
- `'replace'`: replace one existing function in the accepted baseline program.
Use this for `'predict_next_state'` and for any function that already exists, including nested functions.
- `'add'`: add one new top-level function whose name does not exist when this edit is applied.
Its `'code'` must be an unindented top-level `'def ...'`; never use it for existing or nested functions.
- `'delete_functions'`: delete existing functions by name before applying `'edits'`.
`'predict_next_state'` cannot be deleted; missing names are ignored, top-level matches are deleted before nested ones, and ambiguous same-level matches are rejected.

Figure 5: Inductive program update prompt template.

A.3 Information-Theoretic View of Frontier Scoring

This section gives an information-theoretic view of the frontier score used by the Explorer. The score is designed to make frontier coverage program-relative: the Explorer should cover frontier candidates in a way that is informative about the hypothesis classes induced by the accepted program-update sequence. Let $C_{\tau_x} \in \mathcal{C}$ denote the hypothesis class assigned to the induced transition τ_x after executing frontier candidate x . This motivates the objective

$$\max_{\pi} H_{\pi}(x) + I_{\pi}(C_{\tau_x}; x).$$

The entropy term $H_{\pi}(x)$ expands coverage over frontier candidates, but by itself it may only increase surface variation among state-action pairs. The mutual-information term $I_{\pi}(C_{\tau_x}; x)$ makes this coverage class-informative: when x carries information about C_{τ_x} , increasing coverage over x is aligned with increasing coverage over hypothesis-class evidence.

This form follows two ideas from reward-free exploration and skill discovery. APT motivates entropy-based coverage as a reward-free exploration objective [19]. DIAYN and CIC motivate mutual information as a way to make a diversity variable recoverable from generated experience or learned representations [5, 16]. Alice changes the role of this variable: C_{τ_x} is not a skill, option, or behavior variable to be executed, but a hypothesis class induced by accepted program updates. The purpose is therefore not to learn a reusable skill repertoire, but to make frontier coverage relative to the current executable program.

Because $x = (s, a)$ is a structured state-action object, we formulate the criterion in a learned representation space, with $h = h_{\theta}(x)$. The representation-level objective is

$$\max_{\pi, \theta} H_{\pi}(h) + I_{\pi}(C_{\tau_x}; h).$$

The role of h_{θ} is to preserve hypothesis-class distinctions while providing a space in which coverage can be estimated.

Using the mutual-information identity,

$$I_{\pi}(C_{\tau_x}; h) = H_{\pi}(C_{\tau_x}) - H_{\pi}(C_{\tau_x} | h),$$

we obtain

$$J_{\text{explore}} = H_{\pi}(h) - H_{\pi}(C_{\tau_x} | h) + H_{\pi}(C_{\tau_x}).$$

Maximizing this objective imposes three pressures. The first term, $H_{\pi}(h)$, encourages coverage in the learned embedding space. The second term, $-H_{\pi}(C_{\tau_x} | h)$, makes the embedding class-informative. The final term, $H_{\pi}(C_{\tau_x})$, encourages coverage across hypothesis classes, preventing the policy from satisfying the objective by expanding variation inside only one or a few classes.

The frontier score in Section 4.2 is a computable approximation of this criterion. The embedding-novelty score $r_h(x)$ approximates the $H_{\pi}(h)$ term, the active-class contrastive objective supports the $-H_{\pi}(C_{\tau_x} | h)$ term, and the expected class-coverage score $r_{\mathcal{C}}(x)$ approximates the $H_{\pi}(C_{\tau_x})$ term. Thus,

$$S_{\text{frontier}}(x) = \lambda_h r_h(x) + \lambda_{\mathcal{C}} r_{\mathcal{C}}(x)$$

can be viewed as a practical frontier-scoring approximation to the entropy–mutual-information objective.

Table 6: Hypothesis-class alignment for Alice. Purity is measured from heuristic transition classes $\mathcal{C}_H$ to Alice’s learned classes $\mathcal{C}$ on human-play solution transitions.

Setting	Evaluation Set	Purity $\uparrow$	$\#\mathcal{C}_H$	$\#\mathcal{C}$
Default World	Regular levels	0.978	595	23
Default World	Extra levels	0.955	293	14
Baba in Wonderland	Regular levels	0.793	595	66
Baba in Wonderland	Extra levels	0.817	293	36

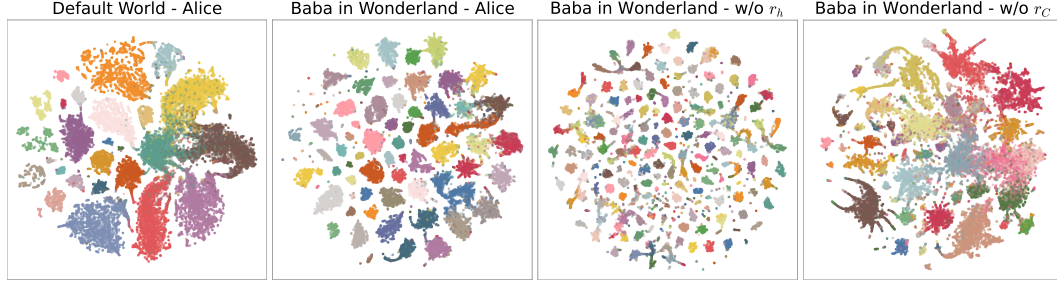


Figure 6: t-SNE visualization of the Explorer’s learned state-action embedding space. Points are explained transitions projected from the 384-dimensional embedding space, and colors denote assigned hypothesis classes. From left to right: Alice in *Default World*, Alice in *Baba in Wonderland*, **w/o** r_h in *Baba in Wonderland*, and **w/o** r_C in *Baba in Wonderland*.

A.4 Additional Experiments

Hypothesis-Class Alignment. We further test whether Alice’s learned hypothesis classes remain tied to transition structure when semantic shortcuts are removed. For this diagnostic, we evaluate Alice’s classes $\mathcal{C}$ on the solution transition dataset and compare them with an evaluation-only heuristic partition $\mathcal{C}_H$, which groups transitions by action and canonical state-difference signature and is never used during learning. We report micro-averaged one-way purity from $\mathcal{C}_H$ to $\mathcal{C}$: for each heuristic class, we assign its evaluated transitions to their majority Alice class and aggregate the majority counts across heuristic classes. The reported $\#\mathcal{C}$ is therefore the number of Alice classes represented among the evaluated solution transitions, not the total number of classes discovered in the broader exploration archive. As shown in Table 6, purity remains substantial in both *Default World* and *Baba in Wonderland*, despite a drop under the misleading Wonderland labels. This indicates that transitions with the same behavioral effect remain largely grouped under Alice’s learned partition, even when the surface rule-property labels no longer provide reliable semantic cues.

Embedding Space t-SNE Visualization. Figure 6 provides a qualitative diagnostic of how Alice’s frontier score shapes the learned state-action embedding space. For each panel, we sample 50,000 explained transitions from a representative run, project their 384-dimensional Explorer embeddings to two dimensions with t-SNE, and color each point by its assigned hypothesis class. Alice forms visible class-structured embeddings in both *Default World* and *Baba in Wonderland*, suggesting that the representation remains organized by transition evidence and preservation conflicts even when surface rule-property labels are misleading. The ablations reveal complementary failure modes. Without embedding novelty (**w/o** r_h), the embedding remains class-aware, but the visited regions are fragmented into small, similarly sized clusters, indicating limited coverage of the broader state-action space. Without expected class coverage (**w/o** r_C), exploration expands more aggressively through the embedding space, but the samples concentrate within a few large hypothesis classes, producing class-imbalanced coverage and over-exploring class-specific variants. Alice balances these pressures: embedding novelty expands the program-relative state-action geometry, while expected class coverage distributes that expansion across underrepresented hypothesis classes near the explanatory frontier.

Coverage Transition Dataset Evaluation. The main experiments evaluate on human-play solution transitions, which concentrate on behavior encountered along successful solution trajectories. As a complementary stress test, we evaluate the online runs trained on regular levels on the coverage transition dataset collected by BFS over each held-out extra level. All Acc. uses the full coverage

Table 7: Evaluation on the held-out extra-level coverage transition dataset. All Acc. is measured on the full coverage transition dataset; Balanced Acc. is measured on the corresponding class-reduced coverage transition set.

Method	Default World		Baba in Wonderland	
	All Acc. $\uparrow$	Balanced Acc. $\uparrow$	All Acc. $\uparrow$	Balanced Acc. $\uparrow$
WorldCoder	0.984	0.894	0.749	0.029
Alice	0.999	0.996	0.997	0.995

Table 8: Offline ablation of Alice’s program-update evidence hyperparameters in *Baba in Wonderland*. The default Alice setting is $n = 3, m = 1$.

Evidence Setting	All Acc. $\uparrow$	Balanced Acc. $\uparrow$	Calls
$n = 1, m = 1$	0.722	0.621	51.0
$n = 1, m = 3$	0.857	0.809	100.0
$n = 3, m = 1$ (Alice)	0.956	0.932	100.0
$n = 3, m = 3$	0.964	0.911	100.0

transition dataset, while Balanced Acc. uses the corresponding class-reduced coverage transition set to discount repeated local effects. Table 7 shows that WorldCoder remains much more reliable in *Default World* than in *Baba in Wonderland*; the gap is especially large under Balanced Acc., indicating that lexical shortcuts can preserve common coverage transitions while failing to recover diverse transition effects under prior misalignment. By contrast, Alice maintains high accuracy on the coverage transition dataset in *Baba in Wonderland*, showing that its learned executable model transfers beyond human-play solution trajectories.

Program-Update Evidence Hyperparameters. Alice constructs the LLM update context by selecting up to n affected hypothesis classes and sampling up to m preservation counterexamples from each selected class. The main experiments use $n = 3$ and $m = 1$. We ablate these values in the offline fixed-dataset setting, where all methods train on fixed regular-level human-play transitions and are evaluated on held-out extra-level human-play transitions under a 100-call cap. Table 8 shows that performance is sensitive to how preservation evidence is packaged. The default $n = 3, m = 1$ setting gives the strongest Balanced Acc., while $n = 3, m = 3$ slightly improves All Acc. at the cost of lower Balanced Acc. The $n = 1$ variants are substantially weaker, indicating that too little class diversity in the program update context can miss rare preservation constraints. We therefore treat these results as a sensitivity analysis: class-structured preservation evidence is useful, but larger update contexts are not automatically better under a fixed LLM-call budget.

LLM Backbone Variation. We evaluate Alice with different LLM backbones while keeping the prompting format, program-update loop, exploration strategy, and evaluation protocol fixed. Table 9 reports online performance on regular-level human-play transitions in *Baba in Wonderland*. The runs show different update dynamics across backbones. In our qualitative inspection, Gemini 3.1 Pro quickly produced a broad program that achieved high accuracy, but later had difficulty revising that program for a few special cases and terminated before using the full call budget. GPT-5.4 instead tended to make more incremental program revisions and continued until the full 100-call budget was exhausted. The smaller-model runs terminated earlier and reached lower accuracy, especially under Balanced Acc. These results suggest that LLM backbones can differ not only in final accuracy, but also in how they revise executable programs; Alice’s class-structured update and exploration loop nevertheless remains usable across the tested backbones.

Equal-Budget WorldCoder Comparison. The main online comparison in Table 1 uses the strict online protocol. Under this protocol, the WorldCoder run in *Baba in Wonderland* stops after 57 calls because it exhausts the 15-call retry budget on one failed program update. We stop there rather than skipping the failed transition: skipping would let the system continue collecting data with a program that is already known not to explain the current frontier transition. We also evaluate a more favorable diagnostic for WorldCoder. In this diagnostic, WorldCoder is allowed to keep collecting interaction evidence until the same 100-call cap used by Alice, giving it more data and more chances to accept

Table 9: Alice with different LLM backbones in *Baba in Wonderland*. Results are online regular-level human-play transition accuracy under the same exploration and update protocol.

Backbone	All Acc. $\uparrow$	Balanced Acc. $\uparrow$	Calls
GPT-5.4-mini	0.865	0.713	41.0
Gemini 3.0 Flash	0.958	0.903	56.0
GPT-5.4	0.982	0.973	100.0
Gemini 3.1 Pro	0.993	0.992	65.0

Table 10: Equal-budget diagnostic in *Baba in Wonderland*. Learned online programs are selected under a 100-call cap and evaluated on regular- and extra-level solution transition datasets.

Method	Regular Levels		Extra Levels		Calls
	All Acc. $\uparrow$	Balanced Acc. $\uparrow$	All Acc. $\uparrow$	Balanced Acc. $\uparrow$	
WorldCoder	0.869	0.755	0.786	0.727	100.0
Alice	0.982	0.973	0.970	0.959	100.0

later updates. We then evaluate the last accepted WorldCoder program before the cap, discarding any unfinished update attempt. As shown in Table 10, this more favorable WorldCoder run improves substantially over the 57-call run, but still remains below Alice on both regular and extra levels, especially under Balanced Acc. This suggests that Alice’s advantage is not only that it avoids early termination, but that its class-structured preservation evidence helps the accepted program cover more distinct transition effects.

A.5 Qualitative Examples

For readability, the visualizations in this subsection render states with the original *Baba Is You* sprites, although the evaluated transitions and learned programs come from *Baba in Wonderland*; only the textual rule-property labels are remapped in the agent’s observations.

Success Cases. Figure 7 shows an evaluation transition on which Alice predicts the exact next state while all three baselines fail. In the underlying game semantics, BABA IS YOU, ROCK IS PUSH, and JELLY IS PUSH are active; in *Baba in Wonderland*, these appear as BABA IS STRANGE, ROCK IS GROW, and JELLY IS GROW. When BABA moves right, the correct transition is a two-object push chain: JELLY moves into the empty cell, ROCK moves into the JELLY’s previous cell, and BABA moves into the ROCK’s previous cell. Alice’s learned program succeeds because it infers that the surface label GROW plays the role of PUSH and applies a recursive chain movement routine before moving the controlled object. The selected WorldCoder program moves controlled objects but does not model this push-chain effect, while the GIF-MCTS and CWM programs contain generic push logic but do not identify the Wonderland GROW property as PUSH in this configuration. All three baselines therefore leave the ROCK–JELLY chain behind, whereas Alice recovers the full transition from evidence despite the misleading property label.

Figure 8 shows a different kind of success case involving simultaneous movement and blocking. Here BABA IS YOU and ROCK IS YOU make both BABA and ROCK controlled objects, while HEDGE IS STOP makes HEDGE blocking objects; in *Baba in Wonderland*, these rules appear as BABA IS STRANGE, ROCK IS STRANGE, and HEDGE IS EAT. After the upward action, the lower controlled objects move up, but the upper ROCK is immediately below a HEDGE and should therefore remain in place while updating its facing direction. Alice’s learned program handles this mixed outcome: it moves each controlled object in the correct order, checks the blocking constraint for the ROCK adjacent to the HEDGE, and preserves the blocked ROCK rather than deleting it or moving it into the HEDGE cell. The baselines miss this interaction in different ways: WorldCoder lacks the corresponding stop-blocking rule, GIF-MCTS omits the blocked ROCK, and CWM moves the ROCK into the blocked cell. This case illustrates why balanced transition accuracy is diagnostic: the difficult part is not moving a controlled object in isolation, but preserving the exact simulator semantics when several controlled objects interact with blocking structure at the same time.

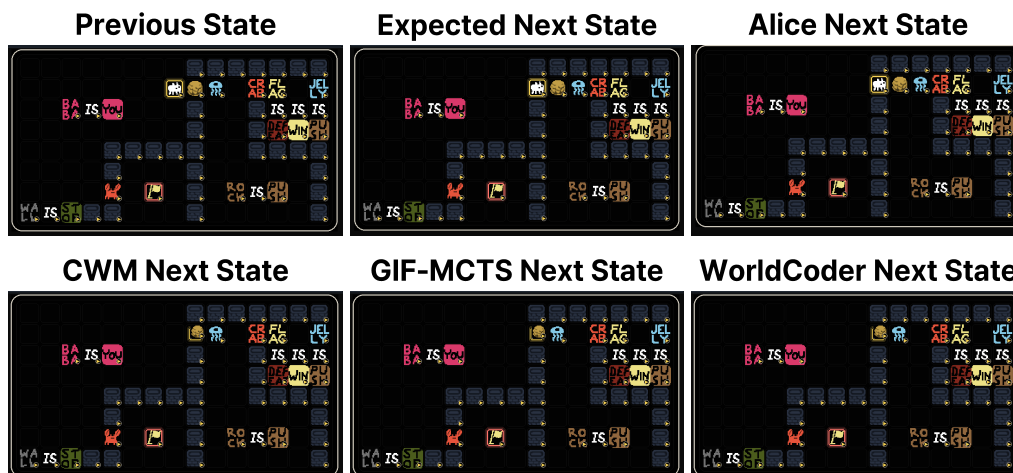


Figure 7: Success Case 1 from *Baba in Wonderland*: a push chain under misleading rule-property labels. Each method panel shows the previous state, expected next state, and the method’s predicted next state.

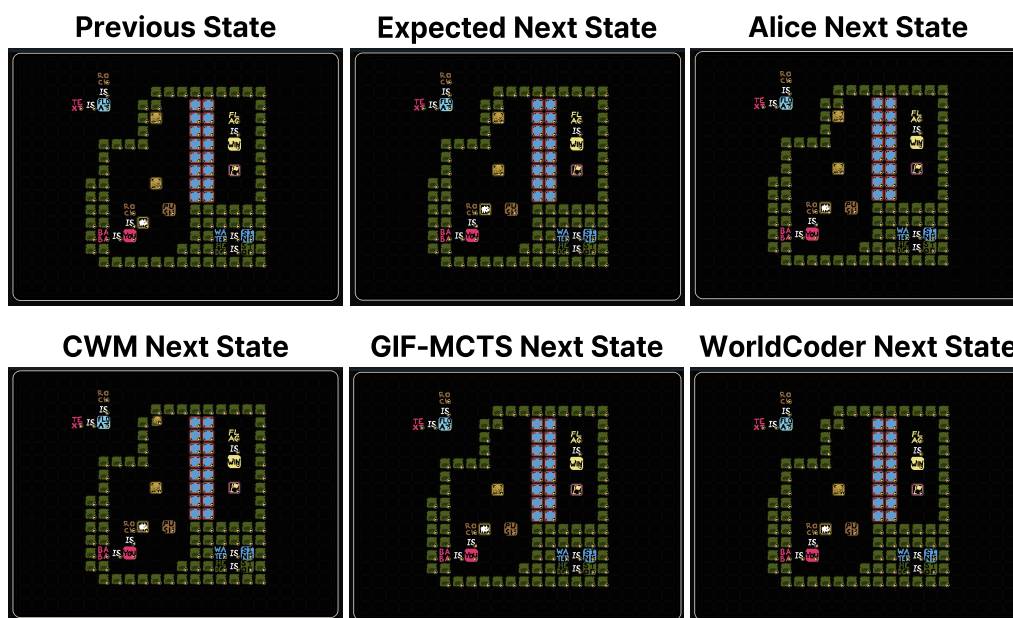


Figure 8: Success Case 2 from *Baba in Wonderland*: simultaneous movement of multiple YOU objects with one controlled object blocked by a stop object. Each method panel shows the previous state, expected next state, and the method’s predicted next state.

Failure Cases. A representative failure case is shown in the top row of Figure 9. In this configuration, WALL IS SHUT, STAR IS OPEN, and STAR IS PUSH are active; in *Baba in Wonderland*, these appear as WALL IS LATE, STAR IS MAD, and STAR IS GROW. When BABA pushes the KEY text block, the pushed block in turn displaces the STAR into the WALL. Under the simulator dynamics, the resulting OPEN–SHUT interaction should remove both the STAR and the WALL, even though the STAR was moved indirectly rather than directly by a YOU object. Alice’s learned program fails to predict this cancellation and leaves the objects in place. The error comes from an over-simplified collision routine in the induced program: because many important interactions in the data are triggered by agent-controlled movement, the learned code checks object–object interaction effects primarily for



Figure 9: Failure case studies from *Baba in Wonderland*.

motion initiated by YOU-controlled objects. In this case, however, the relevant overlap is created by a STAR that is displaced by another pushed object, not by BABA itself. As a result, the learned program misses the OPEN–SHUT resolution that the simulator applies, illustrating how Alice can fit a compact but overly narrow causal account of collision dynamics.

A second failure case is shown in the middle row of Figure 9. Here a noun-to-noun transformation rule of the form `PILLAR IS BABA` becomes active, and the learned program correctly converts the PILLAR objects into BABA. However, one of the transformed PILLAR objects occupies a cell already overlapping a STAR object with the DEFEAT property. Under the simulator dynamics, the newly created BABA should then be removed immediately, because BABA is a YOU object and thus dies on overlap with DEFEAT. Alice’s learned program performs the noun-to-noun conversion but does not resolve the resulting world-object interaction afterward, so the transformed BABA incorrectly remains in the predicted next state. This error reflects a highly specific blind spot in the induced program: it models object transformation and object–object collision as partially separate cases, but does not apply collision resolution after a fresh noun-to-noun conversion. Because such transitions are rare in the learning data, the resulting program never acquires the more complete rule that conversion can immediately trigger post-transform interaction effects.

A third failure case is shown in the bottom row of Figure 9. In this configuration, `ROCK IS ROCK` is active when a competing noun-to-noun rule `ROCK IS FLAG` is formed. Under the simulator dynamics, the self-identity rule takes precedence: once `A IS A` is active, it blocks other noun-to-noun rewrites for that object, so the existing ROCK objects should remain ROCK rather than transforming into FLAG. Alice’s learned program instead applies `ROCK IS FLAG` and converts the ROCK objects into FLAG, producing an incorrect next state. This failure reflects a missing rule-ordering constraint in the induced program. From observed transitions alone, it is difficult to infer that self-identity rules are not merely redundant transformations but priority-setting constraints that suppress competing noun conversions. Because this precedence relation appears only in relatively specific configurations, the learned program captures the surface conversion pattern but not the underlying execution order that makes `ROCK IS ROCK` dominate `ROCK IS FLAG`.

Example Update Trace. Figure 10 makes the class-refinement mechanism concrete through one actual update trace from Alice in *Baba in Wonderland*. Alice reaches a transition in the level *Burglary* where `BABA IS STRANGE (YOU)` is active and the action is `right`. The correct next state moves

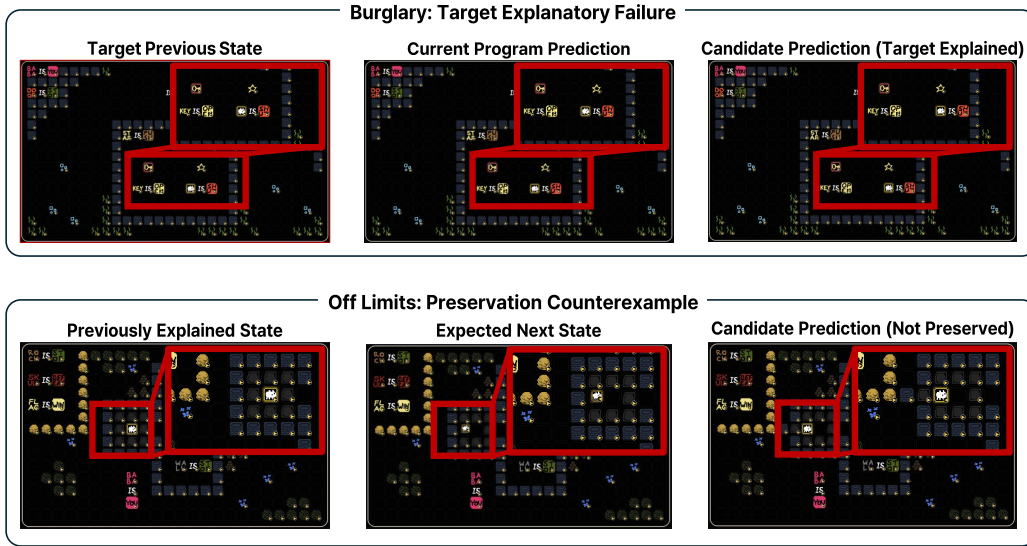


Figure 10: Actual rejected candidate update from Alice in *Baba in Wonderland*. Top: the candidate explains the target transition from *Burglary*, where BABA pushes adjacent rule text. Bottom: the same candidate fails to preserve a previously explained transition from *Off Limits*, exposing a preservation counterexample for the next update prompt.

BABA one cell to the right, but it also moves the adjacent rule-text chain IS LATE one cell to the right. Here LATE is the Wonderland surface label for SHUT. The current program explains ordinary controlled movement, but it does not yet model this case where a controlled object pushes rule text. The transition therefore becomes an explanatory failure.

The first candidate update fixes this target: it moves both BABA and the adjacent text chain. However, the executable acceptance check also tests whether the candidate preserves transitions that the previous program already explained. The candidate fails this check. In a previously explained transition from the level *Off Limits*, BABA IS STRANGE (YOU) is active and the action is `left`. The correct next state moves only BABA one cell left. The candidate instead overgeneralizes the new push rule and also moves nearby non-controlled objects, including a TILE and a WALL. Thus, the candidate has learned a useful part of the target behavior, but it has confused “pushing adjacent rule text” with a broader rule that moves unrelated neighboring objects.

Alice uses this rejection to refine its hypothesis classes. The affected class is split into transitions that the rejected candidate still preserves and transitions that it breaks. The preserved subset contains cases compatible with the candidate’s new behavior; the lost subset contains counterexamples showing where that behavior overgeneralizes. Alice samples a representative lost transition, such as the *Off Limits* case above, and includes it in the next update prompt as preservation evidence. The next candidate is therefore asked to explain the original *Burglary* failure while also preserving the *Off Limits* counterexample. That retry is accepted with no preservation losses. The trace in Figure 10 shows why rejected updates are informative in Alice: they reveal which transitions were incorrectly conflated, split the corresponding hypothesis class, and determine which counterexamples the next LLM update sees.

A.6 Experimental Details

Regular and Extra Levels. We use original *Baba Is You* levels drawn from Stages 1, 2, and 3 of the base game. Following the level taxonomy of the game, we distinguish *regular levels* from *extra levels*; this partition is defined over levels, not by splitting trajectories from the same level. The regular set contains 32 levels: all 8 Stage-1 levels, 13 regular Stage-2 levels, and 11 regular Stage-3 levels. The extra set contains 8 levels: 2 Stage-2 extra levels and 6 Stage-3 extra levels. The

Table 11: Original *Baba Is You* level display names used in the regular and extra level sets.

Stage	Regular Levels	Extra Levels
1	<i>Baba Is You; Where Do I Go?; Now What Is This?; Out Of Reach; Still Out Of Reach; Volcano; Off Limits; Grass Yard</i>	–
2	<i>Icy Waters; Turns; Affection; Pillar Yard; Brick Wall; Lock; Novice Locksmith; Locked In; Changeless; Two Doors; Jelly Throne; Crab Storage; Burglary</i>	<i>Submerged Ruins; Sunken Temple</i>
3	<i>Float; Warm River; Bridge Building; Bridge Building?; Victory Spring; Assembly Team; Catch The Thief!; Tiny Pond; Research Facility; Wireless Connection; Prison</i>	<i>Boiling River; ... Bridges?; Tiny Isle; Dim Signal; Dungeon; Evaporating River</i>

Table 12: Archived solution and coverage transition dataset statistics after canonical transition deduplication.

Split	Dataset	# Levels	# Transitions	# States
Regular Levels	Solution transition dataset	32	2,476	2,490
Regular Levels	Coverage transition dataset	32	3,200,000	1,254,186
Extra Levels	Solution transition dataset	8	1,126	1,126
Extra Levels	Coverage transition dataset	8	800,000	306,488

level display names in each set are listed in Table 11. Online experiments collect interaction data on regular levels and evaluate on human-play solution transitions from those same levels, whereas offline fixed-dataset experiments train on regular-level human-play transitions and evaluate on held-out extra-level human-play transitions.

Dataset Construction. For each level in the regular and extra splits, we construct two transition datasets. The *solution transition dataset* is obtained from successful human solution trajectories and emphasizes the transitions that arise along task-completing behavior. The *coverage transition dataset* is obtained by running BFS on each level and recording canonical transitions until either the reachable frontier is exhausted or 100,000 canonical transitions have been collected for that level. Both datasets are stored as transition archives paired with state archives. Within each level-specific dataset, we canonicalize states and remove duplicate transitions that share the same (s, a, s') triple. This ensures that repeated visits to the same local transition effect do not inflate the archived dataset and that the 100,000-transition cap for coverage is applied to canonical transitions rather than raw observations. Aggregate dataset statistics are summarized in Table 12. The class-reduced subsets used for balanced accuracy are derived from these full transition datasets by the heuristic procedure described in the next subsection.

Evaluation Protocol and Cost Measurement. After training, we evaluate the selected executable world model on the full and class-reduced variants of the solution transition dataset and coverage transition dataset for each split. All Acc. is measured on the full transition dataset, whereas Balanced Acc. is measured on the corresponding class-reduced subset derived by heuristic dynamics discovery. Our primary metric is one-step transition accuracy. For each archived transition (s, a, s') , a prediction is counted as correct only if the program executes successfully on (s, a) and its predicted next state exactly matches the archived next state after canonicalization. Compile failures, contract violations, missing predictions, and runtime errors are all counted as incorrect predictions. We report regular-set and extra-set accuracy separately by aggregating the corresponding archived datasets over levels within each split. For LLM-based methods, we additionally report cumulative training-time LLM cost in terms of Calls.

Compute Resources. Experiments were run on a workstation with an Intel Xeon Gold 6226R CPU at 2.90GHz, 128GB RAM, and two NVIDIA GeForce RTX 3090 GPUs. The operating system was Ubuntu 20.04.6 LTS with Linux kernel 5.4.0-216-generic. A full online run typically required 12–24 hours of wall-clock time in our setup, although runtime varied with external LLM response latency.

Implementation Details. Unless otherwise noted, all online discovery runs use the same experimental pipeline and differ only in the component being ablated. Each run terminates when either the global LLM call budget of 100 is exhausted, a single program-update iteration uses 15 LLM calls without an accepted update, or no candidate update has been accepted for 300,000 global interaction steps. Inductive program update uses a per-target retry cap of 15 rejected candidate updates. When a rejected candidate update causes previously explained transitions to become unexplained, we sample additional transition evidence by selecting up to 3 affected classes uniformly at random and then selecting one broken transition uniformly at random from each selected class. Candidate updates are accepted only if they resolve the target failure and introduce no compile errors, runtime errors, or explanation losses on the full set of previously explained transitions in D_t . For Explorer representation learning, we include a hypothesis class in the active class set only after it has at least 8 assigned transitions, and the learned state-action embedding has dimension 384.

Across LLM-based runs, we keep the prompting and decoding configuration fixed unless the experiment explicitly studies backbone variation. In particular, we use a medium reasoning or thinking setting together with a maximum output budget of 32,768 tokens. In the backbone comparison, we change only the underlying LLM while keeping the update procedure, exploration strategy, prompts, and evaluation protocol unchanged.

A.7 Heuristic Dynamics Discovery for Class-Reduced Subsets

Algorithm. The heuristic dynamics discovery procedure is applied to the archived transition datasets after collection. For each transition, we first reconstruct the source state and next state from the paired state archive. Each state is represented as a multiset of object tokens of the form (type, word, x , y , direction), together with the grid size and termination flags. We then compute a canonical state-difference signature that summarizes how the object multiset changes across the transition.

This signature records three kinds of object-level effects: moved objects, removed objects, and added objects. For moved objects, we match objects by type, word, and direction, and record only the displacement (Δx , Δy) together with the object identity and multiplicity. For objects that disappear or appear without a matched counterpart, we record removal or addition counts by object identity. The signature also records changes in grid size, termination, or truncation when they occur.

Two transitions are assigned to the same heuristic class if and only if they have the same action and the same canonical state-difference signature. As a result, transitions that induce the same local dynamical effect but occur at different absolute positions or in different levels are collapsed into the same heuristic class. This heuristic is not intended to recover the true latent dynamics of the environment. Rather, it provides a simple way to remove large amounts of redundant transition evidence while preserving diversity over distinct observed transition effects.

Subset Construction. Given the heuristic class assignments above, we construct a class-reduced subset by selecting one representative transition from each heuristic class. For each heuristic class, we select one representative transition by uniform random sampling with a fixed seed. The resulting subset therefore contains exactly one transition per heuristic class.

We apply this procedure separately to the solution transition dataset and the coverage transition dataset for each split. The resulting class-reduced subsets are used to compute Balanced Acc. for the solution transition dataset and coverage transition dataset. In our archived datasets, this reduces the train/test solution transition sets from 2,476/1,126 transitions to 595/293 representatives and the train/test coverage transition sets from 3,200,000/800,000 transitions to 7,232/2,073 representatives. These subsets are not intended to be harder benchmarks than the full datasets. Rather, they provide a complementary view that discounts repeated instances of the same local transition effect and emphasizes coverage over distinct heuristic dynamics classes.